

Università degli Studi di Salerno

Corso di Ingegneria del Software

Pick-A-Book Object Design Document Versione 1.4



Data: 26/01/2026

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
Felice Chirico	0512108085
Carmine Nunziata	0512116290
Luca Prisco	0512114127

Scritto da:	Carmine Nunziata - Felice Chirico - Luca Prisco
--------------------	---

Revision History

Data	Versione	Descrizione	Autore
04/12/2025	1.0	prima stesura del documento	Felice Chirico
18/12/2025	1.1	aggiunta delle directory del sistema	Felice Chirico
20/12/2025	1.2	Revisione stile, aggiunto capitolo 4 (Interfacce)	Carmine Nunziata
22/12/2025	1.3	Revisione e aggiunta del capitolo 3 (pacchetti)	Luca Prisco
26/01/2026	1.4	Revisione finale del documento	Carmine Nunziata ,Felice Chirico

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

Indice

Indice	3
1. Introduzione	4
1.1 Object design trade-offs	4
Robustezza vs Tempo	4
Flessibilità vs Semplicità	4
Sicurezza vs Usabilità	4
1.2 Linee guida	5
1.3 Riferimenti	5
2. Directory	5
3. Pacchetti	6
3.1 view	6
3.1.1 view.site	6
3.1.2 view.acquisto	7
3.1.3 view.utente	7
3.1.4 view.catalogo	8
3.2 acquistoManagement	9
3.3 utenteManagement	10
3.4 catalogoManagement	11
4. Interfacce	12
CarrelloIDS	12
GestoreOrdiniIDS	13
OrdineIDS	15
OrdineSingoloIDS	17
CategorialIDS	19
GenerelIDS	19
GestoreCatalogoIDS	20
ProdottoIDS	22
UserIDS	25

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

1. Introduzione

1.1 Object design trade-offs

Nel processo di progettazione del sistema, si è reso necessario operare una serie di compromessi tra differenti qualità del software. Le scelte riportate di seguito tengono conto dei vincoli temporali del progetto, della necessità di rilasciare rapidamente una versione funzionante e della volontà di mantenere un'architettura quanto più chiara e manutenibile nel tempo

Robustezza vs Tempo

La robustezza del sistema dipende fortemente dal livello di validazione degli input e dal numero di controlli effettuati lungo tutto il flusso applicativo. Un controllo completo e rigoroso avrebbe richiesto un tempo di sviluppo maggiore rispetto alla finestra temporale disponibile per il rilascio di una prima versione funzionante del sistema. Per rispettare le scadenze, si è preferito limitare le verifiche approfondite sugli input, mantenendo soltanto quelle essenziali al corretto funzionamento delle funzionalità principali. Il miglioramento dei meccanismi di validazione è previsto nelle versioni successive.

Flessibilità vs Semplicità

Una progettazione altamente flessibile permette ampliamenti futuri con modifiche minime, ma introduce una complessità architettuale più elevata. Considerato l'obiettivo di realizzare rapidamente una prima versione stabile, alcune componenti sono state progettate con un livello di astrazione ridotto, privilegiando:

- Semplicità di lettura del codice;
- Facilità di debugging;
- Rapidità di implementazione.

L'introduzione di estensioni e generalizzazioni strutturate è rinviata alle iterazioni successive del progetto

Sicurezza vs Usabilità

L'introduzione di livelli di sicurezza elevati può comportare controlli aggiuntivi, che a loro volta compromettono la fluidità e la rapidità dell'interazione utente.

Nella fase iniziale si è scelto un livello di sicurezza essenziale, sufficiente per garantire l'autenticazione e l'integrità dei dati sensibili, senza però incidere eccessivamente sull'esperienza d'uso.

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

1.2 Linee guida

- Gli oggetti di tipo DAO devono essere nominati secondo convenzione NomeEntitàIDS;
- I dati in ingresso da parte dell'utente devono essere validati nel layer appropriato e sottoposti ai controlli minimi necessari per la release della prima versione del sistema;
- Il codice del sistema deve rispettare le regole dello standard Java, evitando ridondanze garantendo la chiarezza dei vari moduli del sistema prima delle successive release in cui verrà data all'ottimizzazione del sistema.

1.3 Riferimenti

Per le funzioni descritte in questo documento faremo riferimenti ad altri documenti quali il RAD e il SDD.

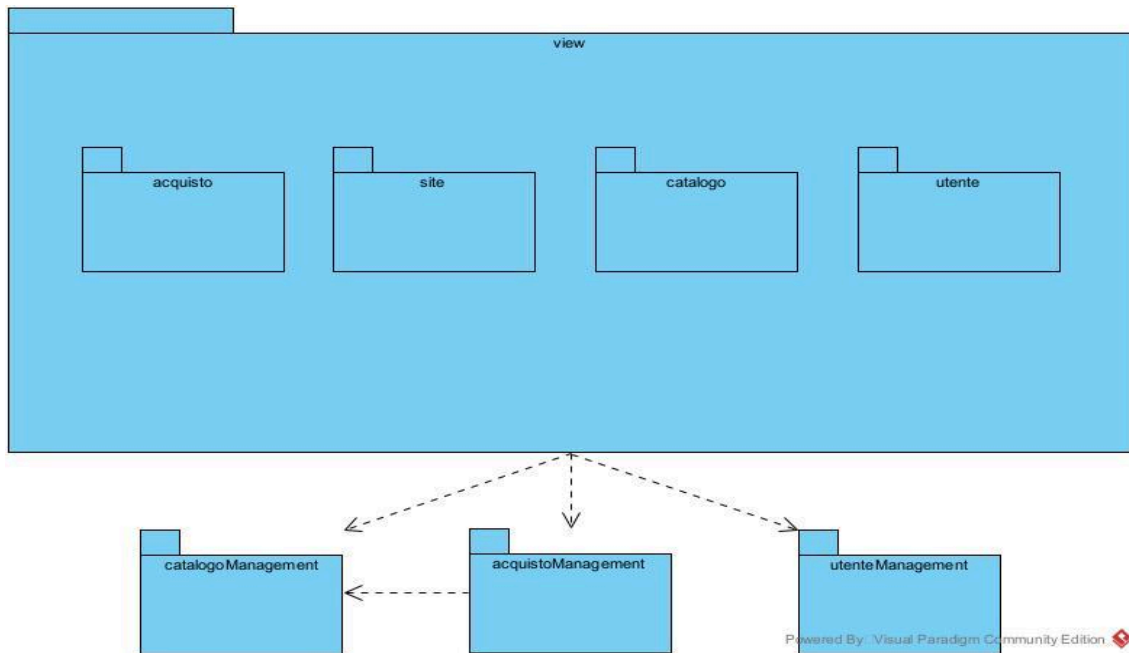
2. Directory

- **src/**
 - java □ Contiene tutti i pacchetti che contengono i codici del sistema.
- **webapp** □ Contiene tutti i pacchetti che contengono i codici del sistema.
- META-INF □ Directory che contiene tutti i file di configurazione.
 - **script** □ Directory che contiene tutti i file javaScript per le funzioni utilizzate dalle jsp.
 - icons □ Directory che contiene tutte le icone e i file PNG relativi alle icone della navbar del sito.
 - images □ Directory che contiene tutte le immagini necessarie per le anteprime dei prodotti.
 - styles □ Directory che contiene tutti i file css relativi allo stile delle diverse pagine del sito.
- **db** □ contiene tutti i gli script necessari per la creazione e il popolamento del database relazionale

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

3. Pacchetti

Il sistema adotta una struttura di package organizzata per garantire una chiara separazione delle responsabilità e facilitare la manutenzione del codice.

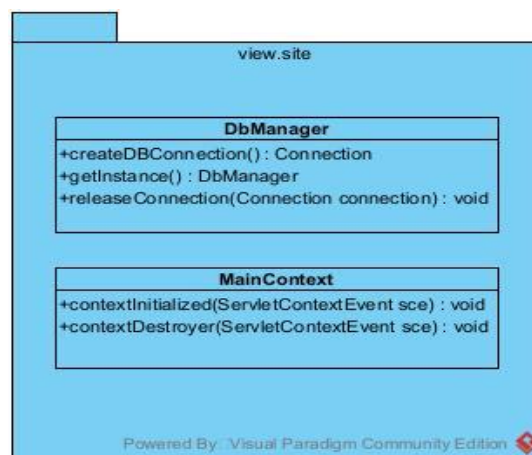


3.1 view

Questo è il pacchetto principale in quanto contiene tutti gli oggetti e le classi Java che compongono il layer di Presentation del sistema.

3.1.1 view.site

Il sottopacchetto `view.site` contiene le classi Java adibite alla connessione al DataBase.

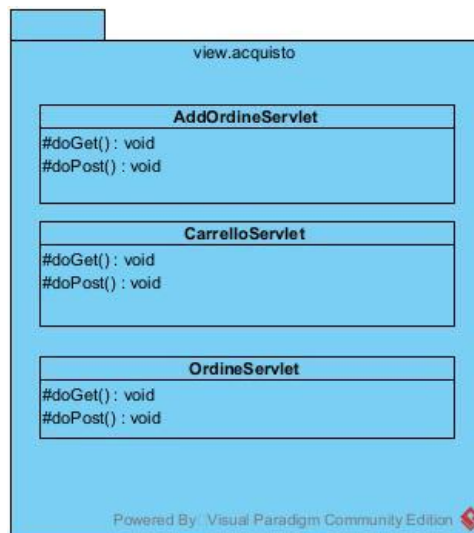


Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

3.1.2 view.acquisto

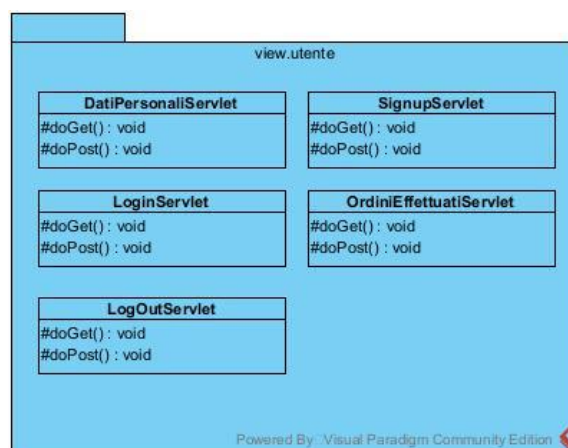
Il sottopacchetto `view.acquisto` contiene le Servlet adibite alle funzioni di acquisto del sito, ad esempio:

- Carrello.
- Pagamento.



3.1.3 view.utente

Il sottopacchetto `view.utente` contiene le Servlet adibite alle funzioni di gestione degli account e di autenticazione al sistema.

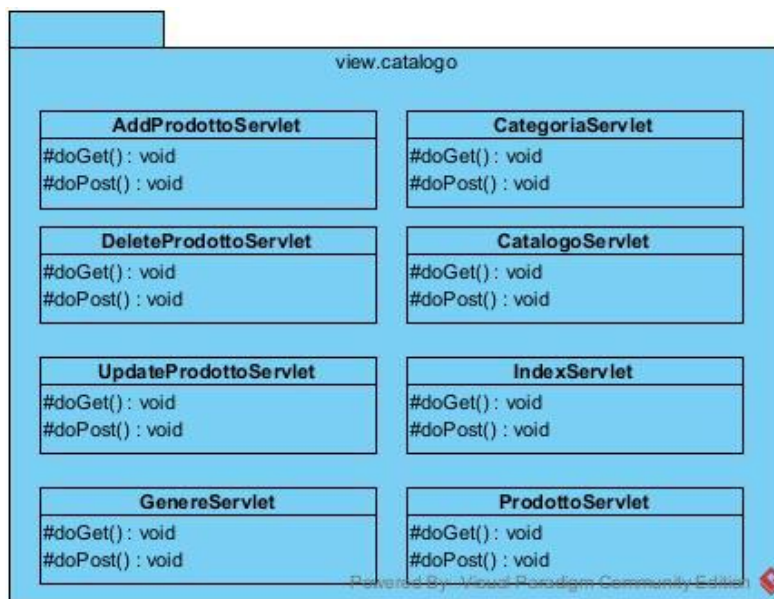


Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

3.1.4 view.catalogo

Il sottopacchetto `view.catalogo` contiene le Servlet adibite alle funzioni di gestione del catalogo, ad esempio:

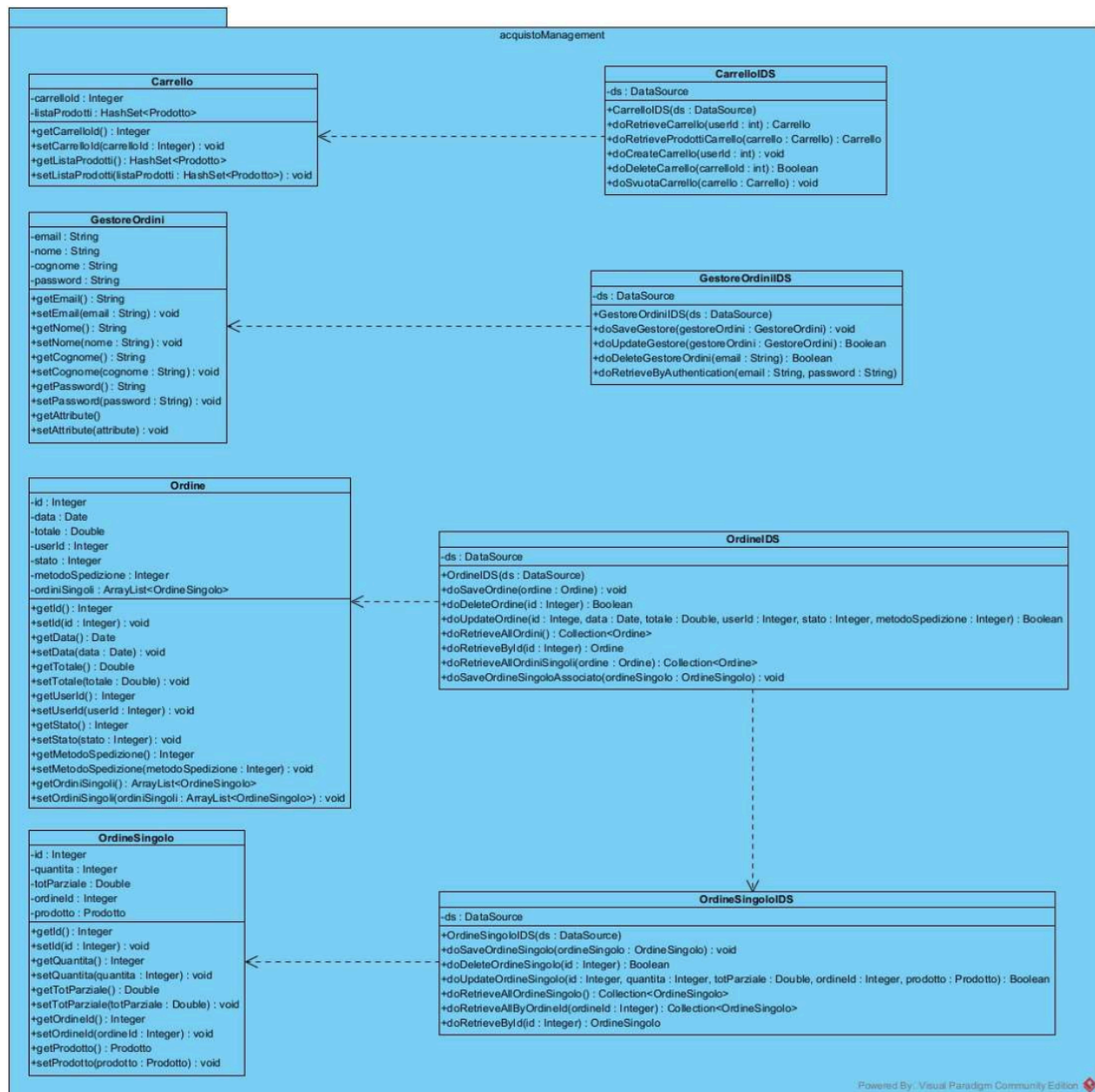
- Visualizzazione del Catalogo.
- Filtraggio dei risultati.



Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

3.2 acquistoManagement

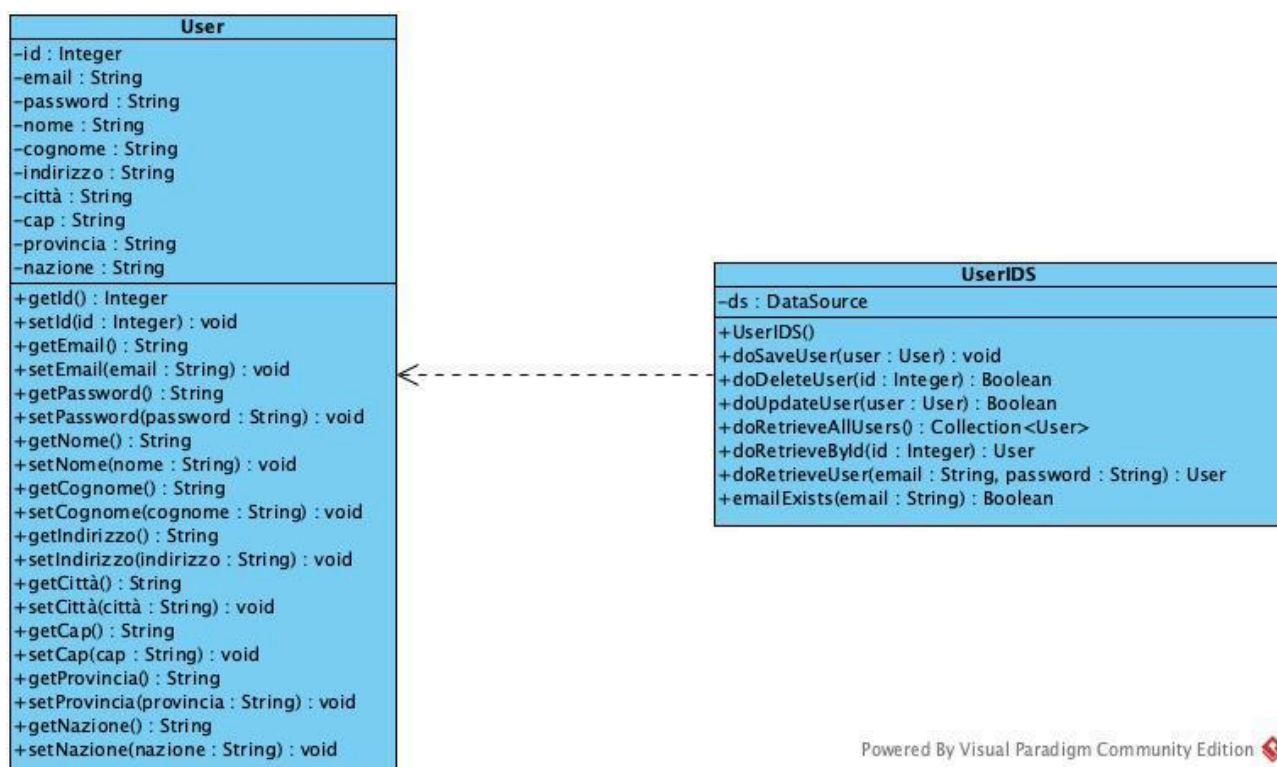
Il sottopacchetto acquistoManagement contiene le classi Java (Bean e/o DAO) adibite alle funzioni di acquisto (ad e.g. la creazione di un nuovo ordine nel DB).



Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

3.3 utenteManagement

Il sottopacchetto `utenteManagement` contiene le classi Java (Bean e/o DAO) adibite alla gestione degli account e delle loro informazioni.

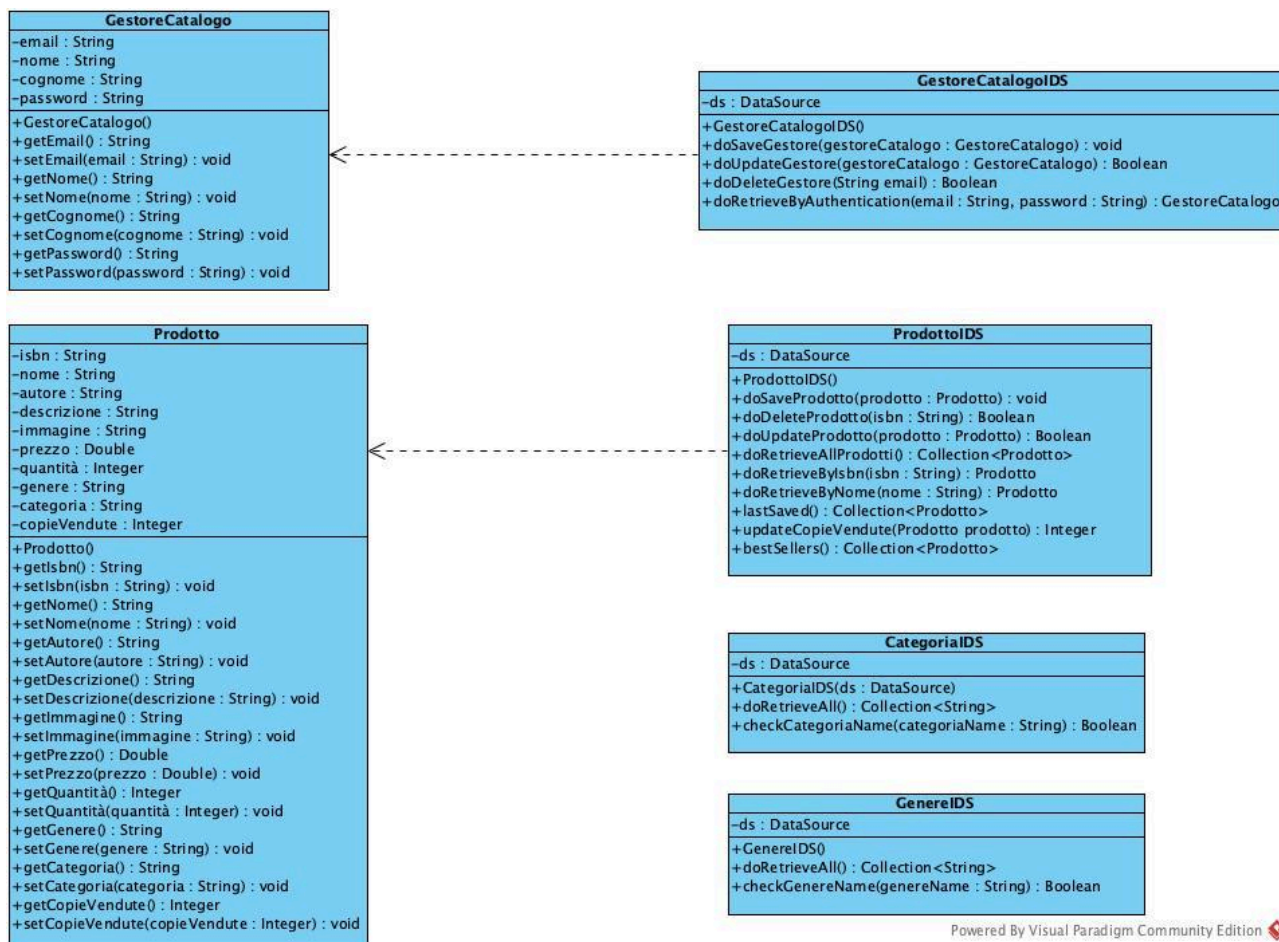


Powered By Visual Paradigm Community Edition

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

3.4 catalogoManagement

Il sottopacchetto `catalogoManagement` contiene le classi Java (Bean e/o DAO) adibite alla gestione del catalogo (informazioni prodotto, dove sono immagazzinati..).



Powered By Visual Paradigm Community Edition

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

4. Interfacce

CarrelloIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità Carrello.
PRECONDIZIONE	<pre>context CarrelloIDS::doCreateCarrello(userId: Integer) pre: userId <> null and userId > 0 context CarrelloIDS::doDeleteCarrello(carrelloId: Integer) pre: carrelloId <> null and carrelloId > 0 context CarrelloIDS::doRetrieveCarrello(userId: Integer) pre: userId <> null and userId > 0 context CarrelloIDS::doRetrieveProdottoCarrello(carrello: Carrello) pre: carrello <> null context CarrelloIDS::doSvuotaCarrello(carrello: Carrello) pre: carrello <> null and not carrello.listaProdotto.isEmpty()</pre>
POSTCONDIZIONE	<pre>context CarrelloIDS::doCreateCarrello(userId: Integer) post: Carrello.allInstances()->exists(c c.userId = userId) and Prodotto.allInstances()->forall(p CarrelloProdotto.allInstances()->exists(cp cp.prodotto = p and cp.carrello.userId = userId)) context CarrelloIDS::doDeleteCarrello(carrelloId: Integer) post: not Carrello.allInstances()->exists(c c.id = carrelloId) and not CarrelloProdotto.allInstances()->exists(cp cp.carrello.id = carrelloId) and result = (Carrello.allInstances()->forall(c c.id <> carrelloId) and CarrelloProdotto.allInstances()->forall(cp cp.carrello.id <> carrelloId)) context CarrelloIDS::doRetrieveCarrello(userId: Integer) post: Carrello.allInstances()->exists(c c.userId = userId) and (if Carrello.allInstances()->exists(c c.userId = userId) then result.userId = userId else result.listaProdotto->isEmpty() endif)</pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

CarrelloIDS	
INVARIANTE	

GestoreOrdiniIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità GestoreOrdini.
PRECONDIZIONE	<pre> context GestoreOrdiniIDS::doSaveGestore (gestoreOrdini: GestoreOrdini) pre: gestoreOrdini <> null and gestoreOrdini.nome <> null and gestoreOrdini.nome <> "" and gestoreOrdini.cognome <> null and gestoreOrdini.cognome <> "" and gestoreOrdini.email <> null and gestoreOrdini.email <> "" and gestoreOrdini.password <> null and gestoreOrdini.password <> "" context GestoreOrdiniIDS::doUpdateGestore (gestoreOrdini: GestoreOrdini) pre: gestoreOrdini <> null and gestoreOrdini.nome <> null and gestoreOrdini.cognome <> null and gestoreOrdini.email <> null and gestoreOrdini.email <> "" and gestoreOrdini.password <> null and gestoreOrdini.password <> "" context GestoreOrdiniIDS::doDeleteGestore (email: String) pre: email <> null and email <> "" context GestoreOrdiniIDS::doRetrieveByAuthentication (email: String, password: String) pre: email <> null and email <> "" and password <> null and password <> "" </pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

GestoreOrdiniIDS	
POSTCONDIZIONE	<pre> context GestoreOrdiniIDS::doSaveGestore (gestoreOrdini: GestoreOrdini) post: GestoreOrdini.allInstances()->exists(go go = gestoreOrdini) context GestoreOrdiniIDS::doUpdateGestore (gestoreOrdini: GestoreOrdini) post: GestoreOrdini.allInstances()->exists(go go.email = gestoreOrdini.email) and result = (GestoreOrdini.allInstances()->forAll(go go.email <> gestoreOrdini.email) and GestoreOrdini.allInstances()->exists(go go = gestoreOrdini)) context GestoreOrdiniIDS::doDeleteGestore (email: String) post: result = (GestoreOrdini.allInstances()->forAll(go go.email <> email) and GestoreOrdini.allInstances()->exists(go go.email = email)) context GestoreOrdiniIDS::doRetrieveByAuthentication (email: String, password: String) post: (GestoreOrdini.allInstances()->exists(go go.email = email and go.password = password) implies result = GestoreOrdini.allInstances()->any(go go.email = email and go.password = password)) and (not GestoreOrdini.allInstances()->exists(go go.email = email and go.password = password) implies result = null) </pre>
INVARIANTE	

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

OrdineIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità Ordine.
PRECONDIZIONE	<pre> context OrdineIDS::doSaveOrdine(ordine: Ordine) pre: ordine <> null and ordine.id > 0 and ordine.data <> null and ordine.totale > 0.0 and ordine.userId > 0 and ordine.stato >= 1 and ordine.stato <= 3 and ordine.metodoSpedizione >= 1 and ordine.metodoSpedizione <= 3 and not ordine.ordiniSingoli->isEmpty() context OrdineIDS::doDeleteOrdine(id: Integer) pre: id > 0 context OrdineIDS::doUpdateOrdine(id: Integer, data: Date, totale: Double, userId: Integer, stato: Integer, metodoSpedizione: Integer) pre: id > 0 and data <> null and totale > 0.0 and userId > 0 and stato >= 1 and stato <= 3 and metodoSpedizione >= 1 and metodoSpedizione <= 3 context OrdineIDS::doRetrieveAllOrdini() pre: true context OrdineIDS::doRetrieveById(id: Integer) pre: id > 0 context OrdineIDS::doRetrieveAllOrdiniSingoli(ordine: Ordine) pre: ordine <> null and ordine.id > 0 context OrdineIDS::doSaveOrdineSingoloAssociato(ordineSingolo: OrdineSingolo) pre: ordineSingolo <> null and ordineSingolo.id > 0 </pre>
POSTCONDIZIONE	<pre> context OrdineIDS::doSaveOrdine(ordine: Ordine) post: Ordine.allInstances()->exists(o o = ordine and ordine.ordiniSingoli->forAll(os </pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

OrdineIDS	
	OrdineSingolo.allInstances()->exists(osi osi = os))) context OrdineIDS::doDeleteOrdine(id: Integer) post: result = (Ordine.allInstances()->forall(o o.id <> id) and not Ordine.allInstances()->exists(o o.id = id)) context OrdineIDS::doUpdateOrdine(id: Integer, data: Date, totale: Double, userId: Integer, stato: Integer, metodoSpedizione: Integer) post: result = (Ordine.allInstances()->exists(o o.id = id and o.data = data and o.totale = totale and o.userId = userId and o.stato = stato and o.metodoSpedizione = metodoSpedizione)) context OrdineIDS::doRetrieveAllOrdini() post: result = Ordine.allInstances() context OrdineIDS::doRetrieveById(id: Integer) post: (Ordine.allInstances()->exists(o o.id = id) implies result = Ordine.allInstances()->any(o o.id = id)) and (not Ordine.allInstances()->exists(o o.id = id) implies result = null) context OrdineIDS::doRetrieveAllOrdiniSingoli(ordine: Ordine) post: result = ordine.ordiniSingoli context OrdineIDS::doSaveOrdineSingoloAssociato(ordineSingolo: OrdineSingolo) post: OrdineSingolo.allInstances()->exists(os os = ordineSingolo)
INVARIANTE	

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

OrdineSingoloIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità OrdineSingolo.
PRECONDIZIONE	<pre> context OrdineSingoloIDS::doSaveOrdineSingolo(ordineSingolo: OrdineSingolo) pre: ordineSingolo <> null and ordineSingolo.id > 0 and ordineSingolo.quantita > 0 and ordineSingolo.totParziale > 0.0 and ordineSingolo.ordineId > 0 and ordineSingolo.prodotto <> null context OrdineSingoloIDS::doDeleteOrdineSingolo(id: Integer) pre: id > 0 context OrdineSingoloIDS::doUpdateOrdineSingolo(id: Integer, quantita: Integer, totParziale: Double, prodotto: Prodotto) pre: id > 0 and quantita > 0 and totParziale > 0.0 and ordineId > 0 and prodotto <> null context OrdineSingoloIDS::doRetrieveAllOrdineSingolo() pre: true context OrdineSingoloIDS::doRetrieveAllByOrdineId(ordineId: Integer) pre: ordineId > 0 context OrdineSingoloIDS::doRetrieveById(id: Integer) pre: id > 0 </pre>
POSTCONDIZIONE	<pre> context OrdineSingoloIDS::doSaveOrdineSingolo(ordineSingolo: OrdineSingolo) post: OrdineSingolo.allInstances()->exists(os os = ordineSingolo) context OrdineSingoloIDS::doDeleteOrdineSingolo(id: Integer) post: result = (OrdineSingolo.allInstances()->forAll(os os.id <> id) and not OrdineSingolo.allInstances()->exists(os os.id = id)) </pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

OrdineSingoloIDS	
	<pre> context OrdineSingoloIDS::doUpdateOrdineSingolo(id: Integer, quantita: Integer, totParziale: Double, prodotto: Prodotto) post: result = (OrdineSingolo.allInstances()->exists(os os.id = id and os.quantita = quantita and os.totParziale = totParziale and os.prodotto = prodotto)) context OrdineIDS::doRetrieveAllOrdineSingolo() post: result = OrdineSingolo.allInstances() context OrdineIDS::doRetrieveAllByOrdineId(ordineId: Integer) post: result = OrdineSingolo.allInstances()->select(os os.ordineId = ordineId) context OrdineIDS::doRetrieveById(id: Integer) post: (OrdineSingolo.allInstances()->exists(os os.id = id) implies result = OrdineSingolo.allInstances()->any(os os.id = id)) and (not OrdineSingolo.allInstances()->exists(os os.id = id) implies result = null) </pre>
INVARIANTE	

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

CategorialDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità Categoria.
PRECONDIZIONE	<pre>context CategorialDS::doRetrieveAll()</pre> <pre>pre:</pre> <pre>context CategorialDS::checkCategoriaName(String categoriaName)</pre> <pre>pre: categoriaName != null</pre>
POSTCONDIZIONE	<pre>context CategorialDS:: doRetrieveAll()</pre> <pre>post: restituisce la lista di tutte le categorie presenti nel Database.</pre> <pre>context CategorialDS::checkCategoriaName(String categoriaName)</pre> <pre>post: controlla che la stringa categoriaName corrisponda ad una delle categorie nel Database.</pre> <pre>Restituisce true se l'operazione è andata a buon fine, altrimenti false</pre>
INVARIANTE	

GenereIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità Genere.
PRECONDIZIONE	<pre>context GenereIDS:: doRetrieveAll()</pre> <pre>pre: true</pre> <pre>context GenereIDS::checkGenereName(genereName: String)</pre> <pre>pre: genereName != null</pre>
POSTCONDIZIONE	<pre>context GenereIDS::doRetrieveAll()</pre> <pre>post: result = Genere.allInstances()</pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

GenereIDS	
	<pre>context GenereIDS::checkGenereName(genereName: String) post: result = Genere.allInstances() ->exists(g g.nome = genereName)</pre>
INVARIANTE	

GestoreCatalogoIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità GestoreCatalogo.
PRECONDIZIONE	<pre>context GestoreCatalogoIDS::doSaveGestore(gestoreCatalogo: GestoreCatalogo) pre: gestoreCatalogo <> null and gestoreCatalogo.nome <> null and gestoreCatalogo.nome <> "" and gestoreCatalogo.cognome <> null and gestoreCatalogo.cognome <> "" and gestoreCatalogo.email <> null and gestoreCatalogo.email <> "" and gestoreCatalogo.password <> null and gestoreCatalogo.password <> "" context GestoreCatalogoIDS::doUpdateGestore(gestoreCatalogo: GestoreCatalogo) pre: gestoreCatalogo <> null and gestoreCatalogo.nome <> null and gestoreCatalogo.cognome <> null and gestoreCatalogo.email <> null and gestoreCatalogo.email <> "" and gestoreCatalogo.password <> null and gestoreCatalogo.password <> "" context GestoreCatalogoIDS::doDeleteGestore(email: String) pre: email <> null and email <> ""</pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

GestoreCatalogoIDS	
	<pre> context GestoreCatalogoIDS::doRetrieveByAuthentication(email: String, password: String) pre: email <> null and email <> "" and password <> null and password <> "" </pre>
POSTCONDIZIONE	<pre> context GestoreCatalogoIDS::doSaveGestore(gestoreCatalogo: GestoreCatalogo) post: GestoreCatalogo.allInstances()->exists(gc gc = gestoreCatalogo) context GestoreCatalogoIDS::doUpdateGestore(gestoreCatalogo: GestoreCatalogo) post: GestoreCatalogo.allInstances()->exists(gc gc.email = gestoreCatalogo.email) and result = (GestoreCatalogo.allInstances()->forAll(gc gc.email <> gestoreCatalogo.email) and GestoreCatalogo.allInstances()->exists(gc gc = gestoreCatalogo)) context GestoreCatalogoIDS::doDeleteGestore(email: String) post: result = (GestoreCatalogo.allInstances()->forAll(gc gc.email <> email) and GestoreCatalogo.allInstances()->exists(gc gc.email = email)) context GestoreCatalogoIDS::doRetrieveByAuthentication(email: String, password: String) post: (GestoreCatalogo.allInstances()->exists(gc gc.email = email and gc.password = password) implies result = GestoreCatalogo.allInstances()->any(gc gc.email = email and gc.password = password)) and (not GestoreCatalogo.allInstances()->exists(gc gc.email = email and gc.password = password) implies result = null) </pre>
INVARIANTE	

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

ProdottoIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità Prodotto.
PRECONDIZIONE	<pre> context ProdottoIDS::doSaveProdotto(prodotto: Prodotto) pre: prodotto <> null and prodotto.isbn.size() = 13 and prodotto.nome <> null and prodotto.nome <> "" and prodotto.autore <> null and prodotto.autore <> "" and prodotto.descrizione <> null and prodotto.descrizione <> "" and prodotto.immagine <> null and prodotto.immagine <> "" and prodotto.prezzo > 0 and prodotto.quantita > 0 and prodotto.genere <> null and prodotto.genere <> "" and prodotto.categoria <> null and prodotto.categoria <> "" context ProdottoIDS::doDeleteProdotto(isbn: String) pre: isbn <> null and isbn.size() = 13 context ProdottoIDS::doUpdateProdotto(prodotto: Prodotto) pre: prodotto <> null and prodotto.nome <> null and prodotto.autore <> null and prodotto.descrizione <> null and prodotto.immagine <> null and prodotto.prezzo <> null and prodotto.quantita <> null and prodotto.genere <> null and prodotto.categoria <> null context ProdottoIDS::doRetrieveAllProdotti() pre: true context ProdottoIDS::doRetrieveByIsbn(isbn: String) pre: isbn <> null and isbn.size() = 13 context ProdottoIDS::doRetrieveByNome(nome: String) pre: nome <> null context ProdottoIDS::lastSaved() pre: true context ProdottoIDS::updateCopieVendute(prodotto: Prodotto) pre: prodotto <> null and prodotto.copieVendute <> null context ProdottoIDS::bestSellers() </pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

ProdottoIDS	
	pre: true
POSTCONDIZIONE	<pre> context ProdottoIDS::doSaveProdotto(prodotto: Prodotto) post: Prodotto.allInstances()->exists(p p = prodotto) context ProdottoIDS::doDeleteProdotto(isbn: String) post: result = (Prodotto.allInstances()->forall(p p.isbn <> isbn) and Prodotto.allInstances()->exists(p p.isbn = isbn)) context ProdottoIDS::doUpdateProdotto(prodotto: Prodotto) post: result = (Prodotto.allInstances()->exists(p p.isbn = Prodotto.isbn and p = prodotto)) context ProdottoIDS::doRetrieveAllProdotti() post: result = Prodotto.allInstances() context ProdottoIDS::doRetrieveByIsbn(isbn: String) post: (Prodotto.allInstances()->exists(p p.isbn = isbn) implies result = Prodotto.allInstances()->any(p p.isbn = isbn)) and (not Prodotto.allInstances()->exists(p p.isbn = isbn) implies result = null) context ProdottoIDS::doRetrieveByNome(nome: String) post: (Prodotto.allInstances()->exists(p p.nome = nome) implies result = Prodotto.allInstances()->any(p p.nome = nome)) and (not Prodotto.allInstances()->exists(p p.nome = nome) implies result = null) context ProdottoIDS::lastSaved() post: result = Prodotto.allInstances()->sortedBy(dataCreazione)->firstN(5) context ProdottoIDS::updateCopieVendute(prodotto: Prodotto) post: result = (Prodotto.allInstances()->exists(p p.isbn = prodotto.isbn and p.copieVendute = Prodotto.copieVendute)) context ProdottoIDS::bestSellers() post: result = </pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

ProdottoIDS	
	Prodotto.allInstances()->sortedBy(copieVendute)->lastN(5)
INVARIANTE	

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

UserIDS	
DESCRIZIONE	Questa classe permette di interfacciarsi al DBMS relazionale e, in particolare, di accedere, modificare e interrogare l'entità site_user.
PRECONDIZIONE	<pre> context UserIDS::doSaveUser(user: User) pre: user <> null and user.id > 0 and user.email <> null and user.email <> "" and user.password <> null and user.password <> "" and user.nome <> null and user.nome <> "" and user.cognome <> null and user.cognome <> "" and user.indirizzo <> null and user.indirizzo <> "" and user.citta <> null and user.citta <> "" and user.cap <> null and user.cap.size() = 5 and user.provincia <> null and user.provincia.size() = 2 and user.nazione <> null and user.nazione <> "" context UserIDS::doDeleteUser(id: String) pre: id > 0 context UserIDS::doUpdateUser(user: User) pre: user <> null and user.id > 0 and user.email <> null and user.password <> null and user.nome <> null and user.cognome <> null and user.indirizzo <> null and user.citta <> null and user.cap <> null and user.cap.size() = 5 and user.provincia <> null and user.provincia.size() = 2 and user.nazione <> null context UserIDS::doRetrieveAllUsers() pre: true context UserIDS::doRetrieveById(id: Integer) pre: id > 0 context UserIDS::doRetrieveUser(email: String, password: String) pre: email <> null and password <> null context UserIDS::emailExists(email: String) pre: email <> null </pre>

Progetto: Pick-A-Book	Versione: 1.4
Documento: Object Design	Data: 26/01/2026

UserIDS	
POSTCONDIZIONE	<pre> context UserIDS::doSaveUser(user: User) post: User.allInstances()->exists(u u = user) context UserIDS::doDeleteUser(id: String) post: result = (User.allInstances()->forall(u u.id <> id) and User.allInstances()->exists(u u.id = id)) context UserIDS::doUpdateUser(user: User) post: result = (User.allInstances()->exists(u u.id = user.id and u = user)) context UserIDS::doRetrieveAllUsers() post: result = User.allInstances() context UserIDS::doRetrieveById(id: Integer) post: (User.allInstances()->exists(u u.id = id) implies result = User.allInstances()->any(u u.id = id)) and (not User.allInstances()->exists(u u.id = id) implies result = null) context UserIDS::doRetrieveUser(email: String, password: String) post: (User.allInstances()->exists(u u.email = email and u.password = password) implies result = User.allInstances()->any(u u.email = email and u.password = password)) and (not User.allInstances()->exists(u u.email = email and u.password = password) implies result = null) context UserIDS::emailExists(email: String) pre: result = User.allInstances()->exists(u u.email = email) </pre>
INVARIANTE	